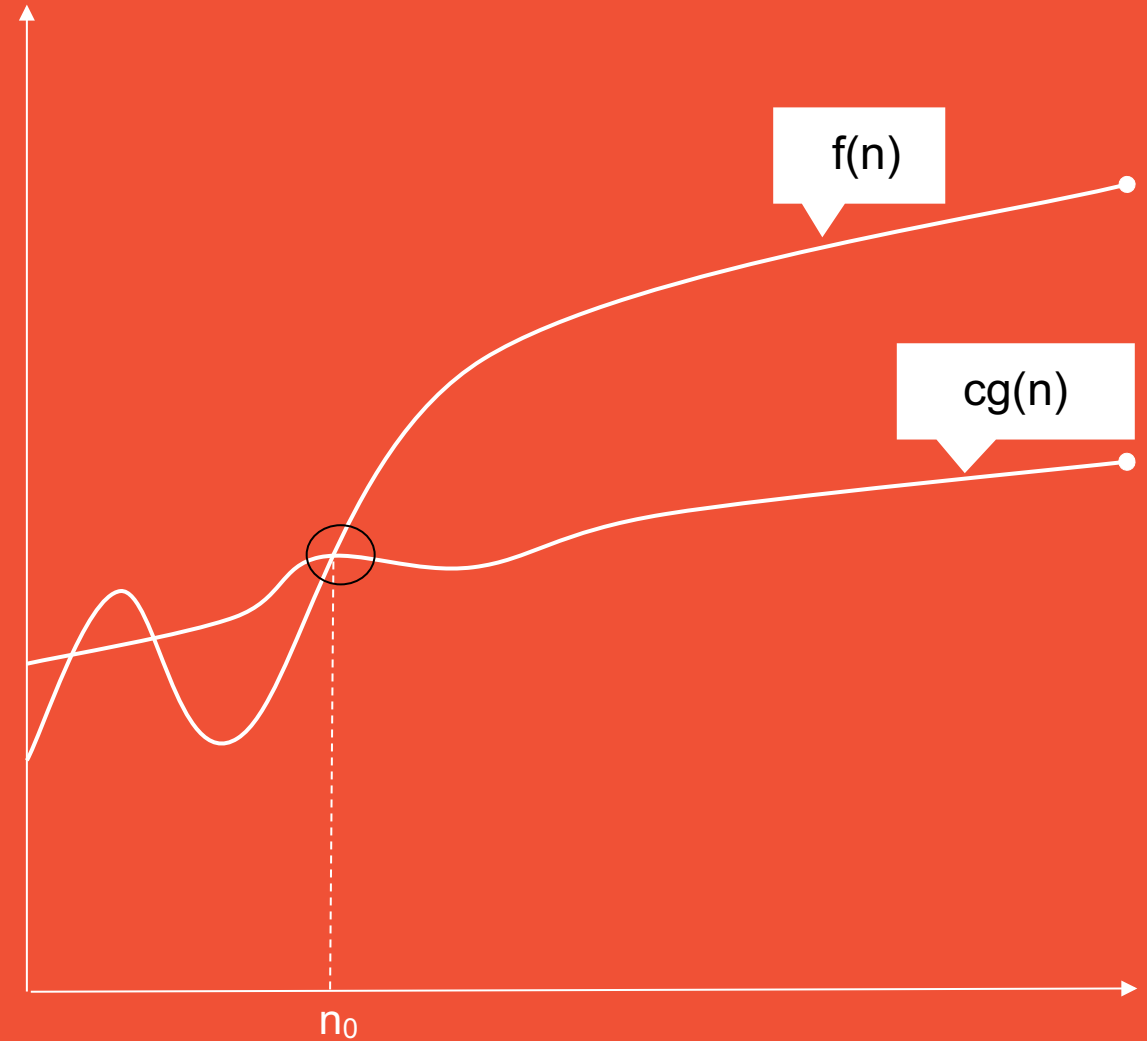


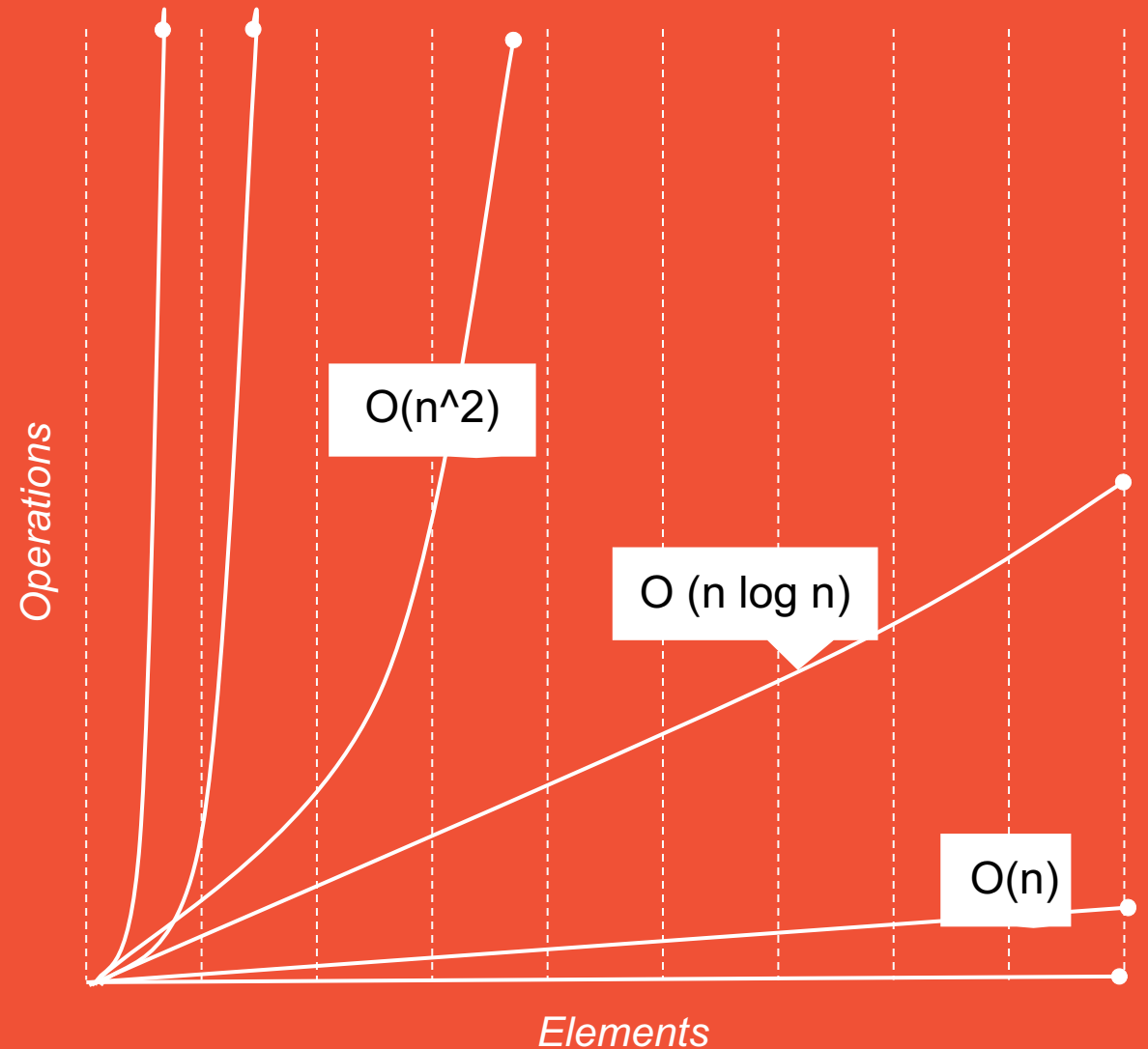


Space Complexity



How To

Calculate Space Complexity



What is Space Complexity?

Space Complexity of an algorithm is the total space taken by the algorithm with respect to the input size.

Space complexity includes both Auxiliary space and space used by input.

Auxiliary Space is the extra space or temporary space used by an algorithm.

Space complexity is a parallel concept to time complexity. If we need to create an array of size n , this will require $O(n)$ space. If we create a two-dimensional array of size $n \times n$, this will require $O(n^2)$ space.

Predict the space complexity for the given below code snippet

```
int add (int n)
{
    if (n <= 0)
    {
        return 0;
    }
    return n + add (n-1);
}
```

A) $O(1)$

B) $O(n)$



C) $O(n * \log n)$

D) $O(n * n)$

Predict the space complexity for the given below code snippet

```
int addSequence (int n)
{
    int sum = 0;
    for(int i = 0; i < n; i++){
        sum += pairSum(i, i+1);
    }
    return sum;
}
int pairSum(int x, int y){
    return x + y;
}
```

A) $O(1)$



B) $O(n)$

C) $O(n * \log n)$

D) $O(n * n)$

Predict the space complexity for the given below code snippet

```
int foo(int n) {  
    int* arr = new int[n];  
    for (int i = 0; i < n; i++) {  
        arr[i] = i;  
    }  
    return arr[n-1];  
}
```

A) $O(1)$

B) $O(n)$



C) $O(n * \log n)$

D) $O(n * n)$

Predict the space complexity for the given below code snippet

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += i;
}
```

A) $O(1)$



B) $O(n)$

C) $O(n * \log n)$

D) $O(n * n)$

Predict the space complexity for the given below code snippet

```
int fibonacci(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    return fibonacci(n-1) + fibonacci(n-  
2);  
}
```

A) $O(1)$

B) $O(n)$



C) $O(n * \log n)$

D) $O(n * n)$

Predict the space complexity for the given below code snippet

```
struct Node {  
    int data;  
    Node* next;  
};  
  
Node* head = nullptr;  
  
void insert(int data) {  
    Node* newNode = new Node();  
    newNode->data = data;  
    newNode->next = head;  
    head = newNode;  
}
```

A) $O(1)$

B) $O(n)$ 

C) $O(n * \log n)$

D) $O(n * n)$



THANK YOU